

DATA WAREHOUSING AND BUSINESS ANALYTICS – GROUP COURSEWORK

Github repository: <https://github.com/rleytonm/DataWarehousing-CW2>

Arora, Paarth

Leyton, Cristian Renato

Mota, Hafsa

TASK A

1. Extracting and storing S&P 500

Installing necessary packages:

```
#Install Package
!pip install yfinance
!pip install yahoofinancials

Show hidden output

import yfinance as yf
from yahoofinancials import YahooFinancials
```

Figure 1 Installation of Yahoo packages

Installation and import of the required Yahoo packages, necessary to extract the S&P 500 financial data.

Figure 2 Web scrapping of the S&P 500 table

Figure 2 shows the use of the “requests” library. This library was used because of Wikipedia blocking the default Pandas user-agent for web scrapping. The code shows how the relevant columns of the table are converted into a dictionary, and later, the ticker list is extracted from the dictionary.

```
# Step added because of Wikipedia blocking pandas user-agent
import requests

url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"

response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
table = pd.read_html(response.text)[0]

# Creating a dictionary with the Symbol and Security columns

sp500_dict = dict(zip(table.iloc[:, 0], table.iloc[:, 1]))

# Extracting the ticker list

tickerlist = list(sp500_dict.keys())

/tmp/ipykernel_17003/1251342058.py:7: FutureWarning: Passing literal html
table = pd.read_html(response.text)[0]
```

```
# Extracting the ticker list

tickerlist = list(sp500_dict.keys())

/tmp/ipykernel_17003/1251342058.py:7: FutureWarning
table = pd.read_html(response.text)[0]

sp500_dict

Show hidden output

tickerlist

['MMM',
 'AOS',
 'ABT',
 'ABBV',
 'ACN',
 'ADBE',
 'AMD',
 'AES',
 'AFL',
 'A',
 'APD',
 'ABNB',
 'AKAM',
 'ALB',
 'ARE',
```

Figure 3 Ticker list

Figure 3 shows a sample of the extracted ticker list.

2. S&P 500 daily performance. Storing the data into a data frame

```
# Define a start date and End Date
start = '2022-01-01'
end = '2025-01-01'

df = yf.download(tickers=tickerlist, start= start , end = end)

Show hidden output
```

df

Price	Adj Close				Close				
Ticker	BF.B	BRK.B	Q	SNDK	A	AAPL	ABBV	ABNB	ABT
Date									
2022-01-03	NaN	NaN	NaN	NaN	151.702148	178.103638	115.847214	172.679993	128.345993
2022-01-04	NaN	NaN	NaN	NaN	146.573685	175.843262	115.624794	170.800003	125.327499
2022-01-05	NaN	NaN	NaN	NaN	144.062744	171.165817	116.232170	162.250000	124.764442
2022-01-06	NaN	NaN	NaN	NaN	144.566895	168.308517	115.684669	159.750000	124.745979
2022-01-07	NaN	NaN	NaN	NaN	140.718048	168.474838	115.385269	166.050003	125.133659
...	...	...	...	...	...	...	...	...	...
2024-12-24	NaN	NaN	NaN	NaN	134.457245	256.797241	172.530685	134.990005	112.058289
2024-12-26	NaN	NaN	NaN	NaN	134.190491	257.612732	171.763870	135.320007	112.556282
2024-12-27	NaN	NaN	NaN	NaN	133.903992	254.201370	170.623230	133.384995	112.282867
2024-12-30	NaN	NaN	NaN	NaN	132.797546	250.829788	168.888351	131.809998	110.144432
2024-12-31	NaN	NaN	NaN	NaN	132.963791	249.059448	170.326096	131.410004	110.447144

753 rows x 2519 columns

Figure 4 Downloading S&P 500 data

Figure 4 shows the variable “df” which stores the information downloaded with the “yf” library, using the ticker list shown in Figure 3. The dates go from 01/01/2022 until 01/01/2025. Dimension of the dataset is 753 rows and 2519 columns.

3. Data preparation

```
# Drop ticker columns where more than 50% of values are missing
threshold = len(df) * 0.5
df = df.dropna(axis=1, thresh=int(threshold))

# Forward fill remaining missing values (carries last known price forward)
df = df.ffill()

# Drop any remaining rows with NaN
df = df.dropna()
```

df

Price	Close							
Ticker	A	AAPL	ABBV	ABNB	ABT	ACGL	ACN	ADBE
Date								
2022-01-03	151.702148	178.103638	115.847229	172.679993	128.345978	42.362530	383.711945	564.369995
2022-01-04	146.573685	175.843246	115.624802	170.800003	125.327484	42.914051	380.969818	554.000000
2022-01-05	144.062759	171.165848	116.232201	162.250000	124.764435	42.410072	374.260773	514.429993
2022-01-06	144.566879	168.308487	115.684669	159.750000	124.745949	42.657307	356.187500	514.119995
2022-01-07	140.718124	168.474854	115.385277	166.050003	125.133614	42.856995	349.355835	510.700012
...	...	...	...	...	...	...	...	...
2024-12-24	134.457245	256.797241	172.530670	134.990005	112.058289	92.669998	356.539642	447.940002
2024-12-26	134.190521	257.612701	171.763855	135.320007	112.556282	92.930000	355.356537	450.160004
2024-12-27	133.903992	254.201370	170.623245	133.384995	112.282875	92.339996	351.166321	446.480011
2024-12-30	132.797531	250.829788	168.888367	131.809998	110.144440	91.889999	347.528290	445.799988
2024-12-31	132.963806	249.059448	170.326111	131.410004	110.447128	92.349998	346.838165	444.679993

753 rows x 2515 columns

Figure 5 Dropping unnecessary columns and null values

Figure 5 shows code to drop any ticker with over 50% of missing information. Also, any null values were replaced by the previous valid value, a common practice in stock market data preparation. Finally, any non-numeric values are dropped from the table. The final dimensionality is 753 rows and 2515 columns.

4. Daily return

```
daily_returns = (df["Close"] - df["Close"].shift(1)) / df["Close"].shift(1)
```

daily_returns								
Ticker	A	AAPL	ABBV	ABNB	ABT	ACGL	ACN	ADBE
Date								
2022-01-03	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2022-01-04	-0.033806	-0.012691	-0.001920	-0.010887	-0.023518	0.013019	-0.007146	-0.018374
2022-01-05	-0.017131	-0.026600	0.005253	-0.050059	-0.004493	-0.011744	-0.017611	-0.071426
2022-01-06	0.003500	-0.016693	-0.004710	-0.015408	-0.000148	0.005830	-0.048291	-0.000603
2022-01-07	-0.026623	0.000988	-0.002588	0.039437	0.003108	0.004681	-0.019180	-0.006652
...	...	...	...	...	...	...	...	...
2024-12-24	0.011144	0.011478	0.009025	0.003494	0.003937	0.006298	0.007972	0.002686
2024-12-26	-0.001984	0.003176	-0.004445	0.002445	0.004444	0.002806	-0.003318	0.004956
2024-12-27	-0.002135	-0.013242	-0.006641	-0.014300	-0.002429	-0.006349	-0.011791	-0.008175
2024-12-30	-0.008263	-0.013263	-0.010168	-0.011808	-0.019045	-0.004873	-0.010360	-0.001523
2024-12-31	0.001252	-0.007058	0.008513	-0.003035	0.002748	0.005006	-0.001986	-0.002512

753 rows x 503 columns

Figure 6 Calculating daily return

Figure 6 shows the application of the Daily Return formula: the closing price of date “t”, minus the closing price of the date “t-1”, divided by the closing price of “t-1”. Since the date “t” of every ticker doesn’t have a “t-1” for reference, the first row is all NaN values. This is a normal behaviour given the context.

5. Beta

```
index = yf.download('^GSPC', start=start, end=end)["Close"].squeeze()
index_returns = index.pct_change().dropna()
index_returns.name = 'GSPC'

# Calculate beta for each stock
betas = {}
for ticker in tickerlist:
    stock_returns = daily_returns[ticker].dropna()
    aligned = pd.concat([stock_returns, index_returns], axis=1, join='inner').dropna()
    cov_matrix = np.cov(aligned[ticker], aligned['GSPC'])
    beta = cov_matrix[0, 1] / np.var(aligned['GSPC'])
    betas[ticker] = beta

betas_df = pd.Series(betas, name='Beta').to_frame()
```

Show hidden output

Betas	
MMM	0.784780
AOS	0.925682
ABT	0.673212
ABBV	0.299496
ACN	1.095665
...	...
XYL	1.069124
YUM	0.634252
ZBRA	1.502187
ZBH	0.737484
ZTS	0.917831

503 rows x 1 columns

Figure 7 Calculating BETA

This code calculates a "Beta" score for 503 stocks to show how much each one swings compared to the overall market. First, it downloads daily price changes for the S&P 500 to use as a benchmark. Then, for every stock in the list, it matches up its daily performance with the market's performance on the same dates. By comparing these movements, the script determines a risk level for each company: for instance, the output shows that ZBRA is highly volatile (1.50), while ABBV is much more stable (0.30).

6. Annual Volatility

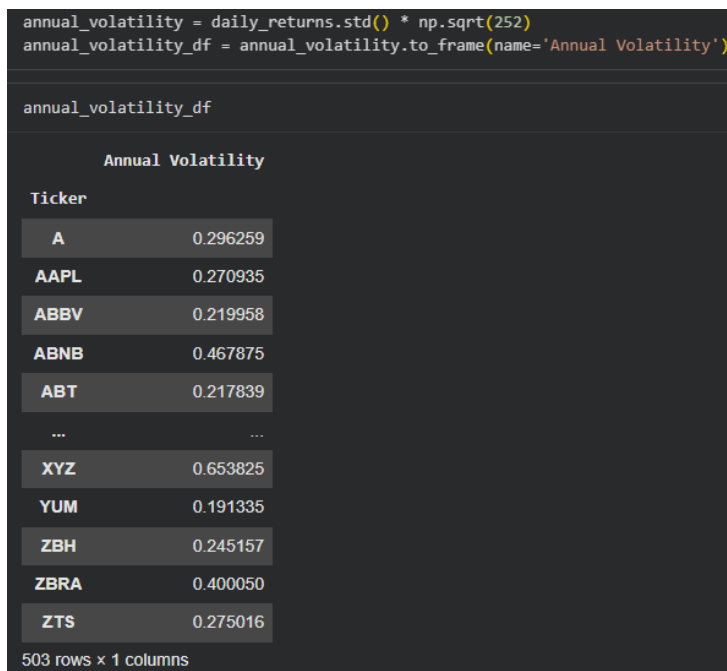


Figure 8 Annual volatility

This code calculates annual volatility, which is a simple way to see how much a stock's price jumps around over a full year. It looks at how much the price changes day-to-day and then scales that number up to cover all 252 trading days in a year. Looking at the results, a stock like YUM (0.19) is relatively steady, while a stock like ABNB (0.46) is much "jumpier" and moves more unpredictably.

7. Visualising the data

a. Daily return

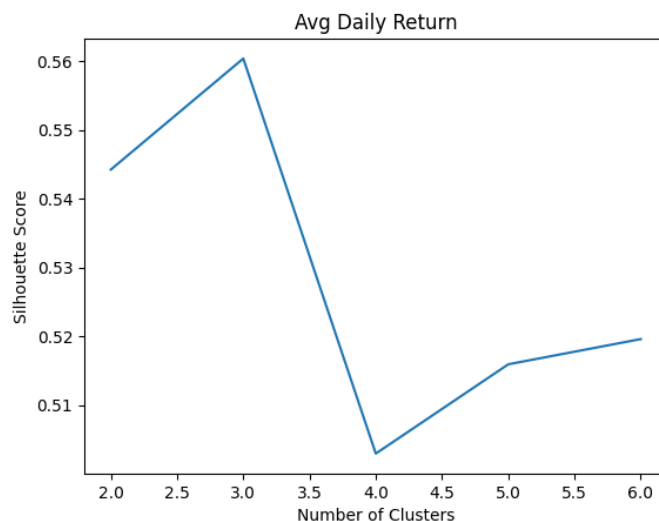


Figure 9 Silhouette score for average daily return

This graph uses the Silhouette Score to determine the most logical number of groups for stocks based on their average daily returns. The highest point on the chart is at 3 clusters, meaning the stocks naturally split into three distinct categories—likely representing low, medium, and high performance.

While grouping them into 2 clusters is also a strong option, the score drops sharply at 4 clusters, indicating that adding more groups creates overlap and makes the categories less meaningful.

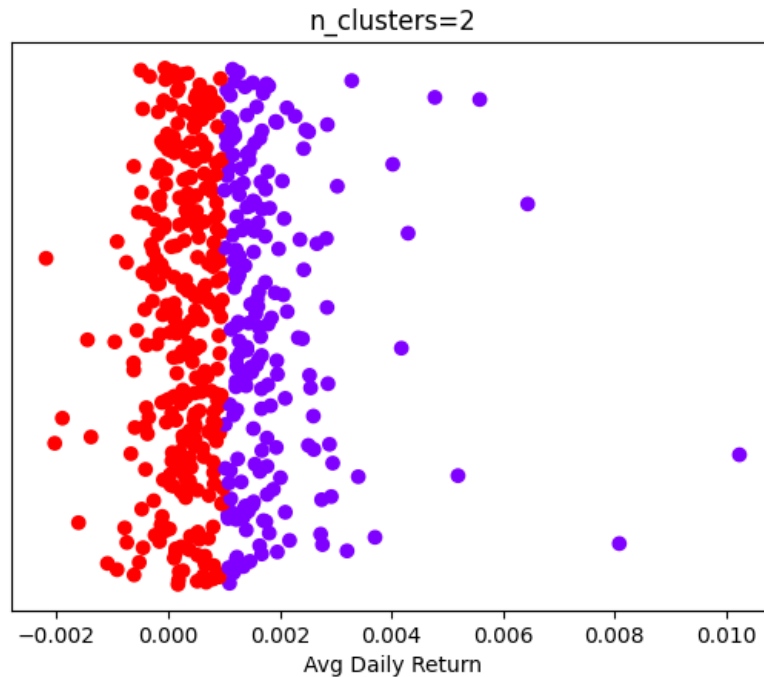


Figure 10 Average daily return with 2 clusters

This scatter plot visualizes the stocks grouped into two clusters based on their average daily returns. The red dots represent a "lower-return" group centred around zero, while the purple dots identify "higher-return" stocks that stretch further to the right. The vertical split between the colours shows that the algorithm has found a clear cutoff point (around 0.001) to distinguish between underperforming and overperforming assets.

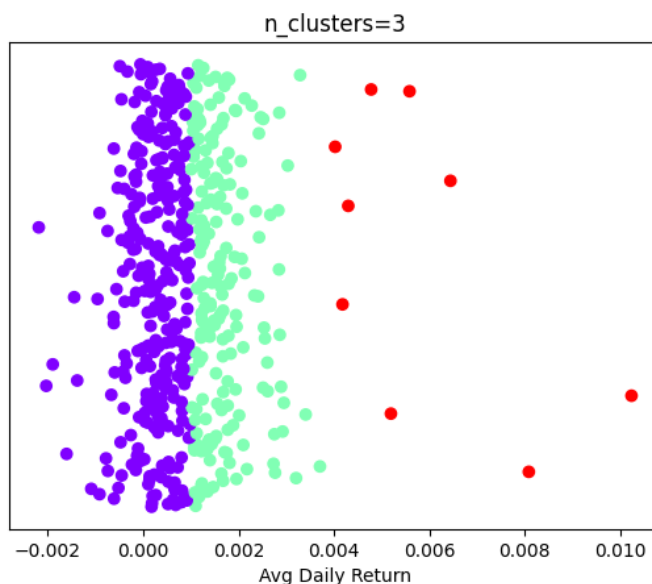


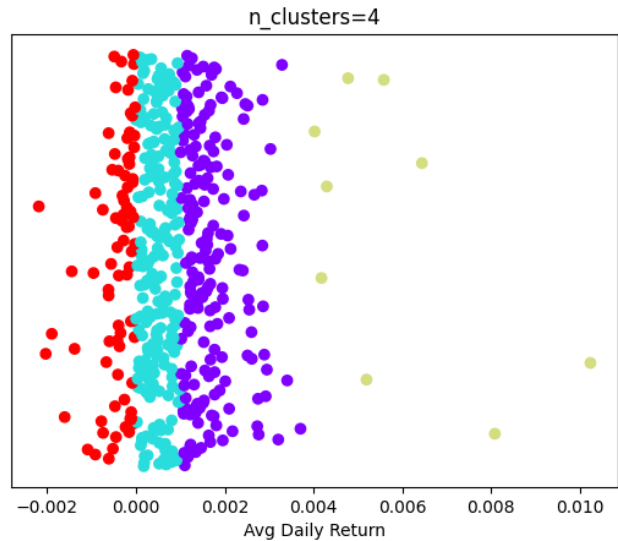
Figure 11 Average daily return with 3 clusters

This chart expands the grouping into three clusters, offering a more detailed look at stock performance. The purple dots represent the largest group of stocks with lower or near-zero returns, while the green dots capture a middle tier of steady gainers. The red dots stand out as a small "high-performer" group of outliers with significantly higher average daily returns, showing

exactly where the top-tier winners are separated from the rest of the market.

Figure 12 Average daily return with 4 clusters

This chart divides the stocks into four clusters, attempting to create even finer distinctions in performance levels. It separates the data into a "negative return" group (red), a "neutral" group (blue), a "positive" group (purple), and the "high performers" (green). However, comparing this to the previous Silhouette Score analysis, these four groups likely overlap too much to be useful, as the data starts to look like narrow vertical strips rather than naturally separate categories.



b. Beta

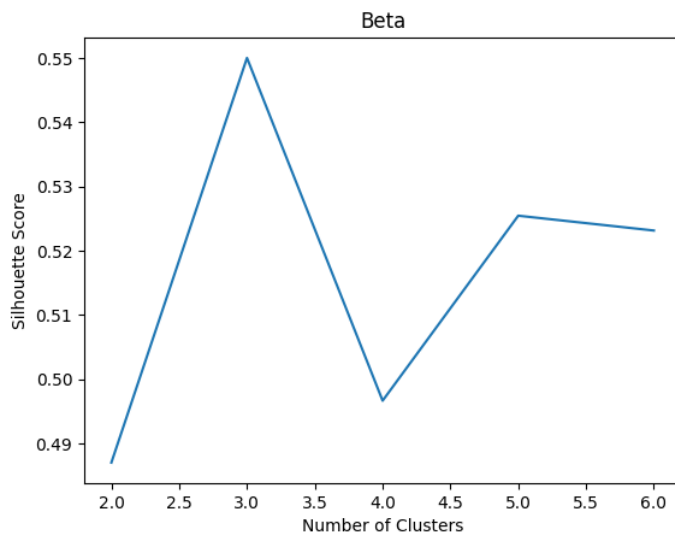


Figure 13 Silhouette score for beta

Figure 13 shows the Silhouette Score to find the most natural groupings for stocks based on their Beta (market risk). The clear peak at 3 clusters indicates that this is the optimal number of groups, likely categorizing stocks into "low," "medium," and "high" risk tiers. While the score drops sharply at 4 clusters, it picks back up slightly at

5, but the 3-cluster model remains the strongest and most distinct way to organize the data.

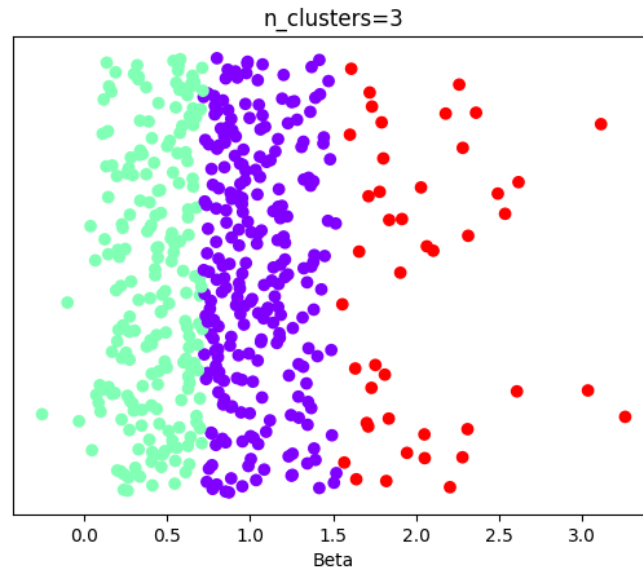


Figure 14 Beta with 3 clusters

This scatter plot visualizes the stocks grouped into three risk categories based on their Beta values. The green dots represent "low-risk" defensive stocks (Beta under 0.75), the purple dots show "market-aligned" stocks (Beta between 0.75 and 1.5), and the red dots identify "high-risk" aggressive stocks that are much more volatile than the market (Beta over 1.5). This grouping confirms that while most stocks stay close to the market average, there is a distinct set of outliers that swing much more wildly.

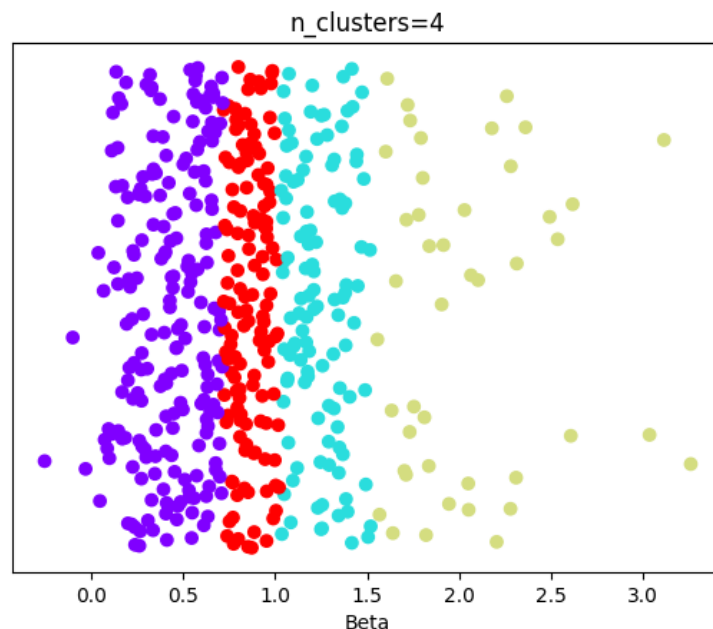


Figure 15 Beta with 4 clusters

Figure 15 shows the stocks divided into four risk groups based on their Beta values, ranging from very stable to highly aggressive. The purple and red groups represent

"defensive" and "market-average" stocks (Beta under 1.0), while the blue and green groups identify increasingly volatile stocks that move more than the general market. However, as noted in Figure 13, using four clusters here creates very thin, overlapping categories that may not be as meaningful as a simpler three-group model.

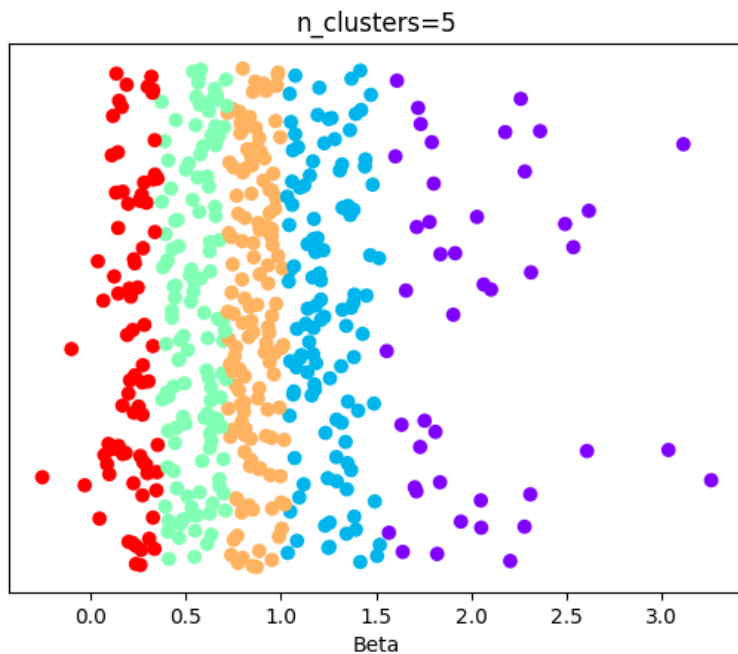


Figure 16 Beta with 5 clusters

This chart breaks the stocks into five distinct clusters to provide a very granular look at market risk. The grouping ranges from "ultra-defensive" stocks in red (Beta near 0) to "extreme-volatility" stocks in purple (Beta above 1.5), with three middle tiers in between. While this five-group model offers more detail, it likely over-segments the data, as many of these categories sit

very close together and lack the clear separation seen in the simpler three-cluster model.

c. Annual volatility

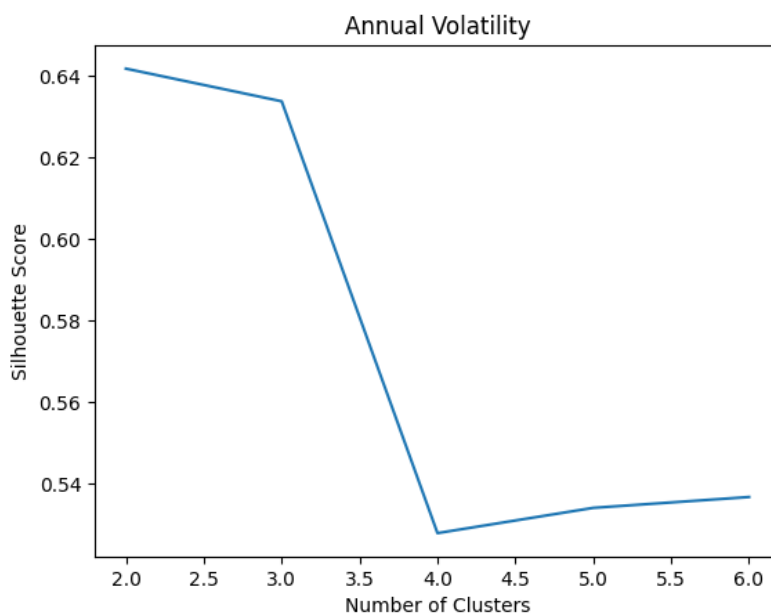


Figure 17 Silhouette score for annual volatility

Figure 17 shows the Silhouette Score to find the most natural way to group stocks based on their Annual Volatility (how much their prices fluctuate). The highest score is at 2 clusters, suggesting that the data most cleanly splits into two simple categories—likely "stable" and "volatile" stocks. While 3 clusters also work well, the score crashes at 4, indicating that trying to create more than three groups leads to messy, overlapping categories that don't represent the data clearly.

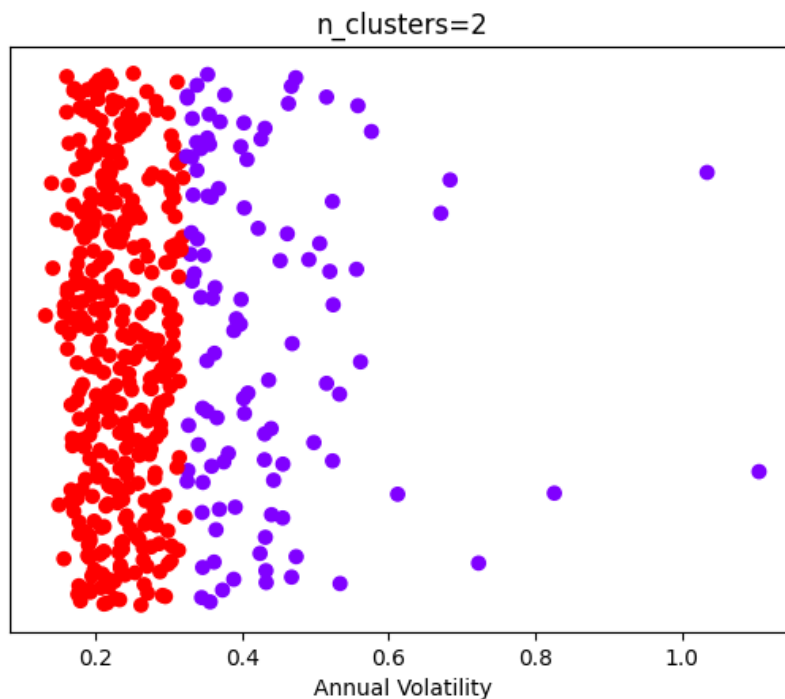


Figure 18 Annual volatility with 2 clusters

This scatter plot shows the stocks divided into two clusters based on their annual volatility, confirming the optimal grouping suggested by the Silhouette Score. The red dots represent a large group of "stable" stocks with lower price fluctuations (mostly below 0.3), while the purple dots identify

"volatile" stocks that experience much larger price swings.

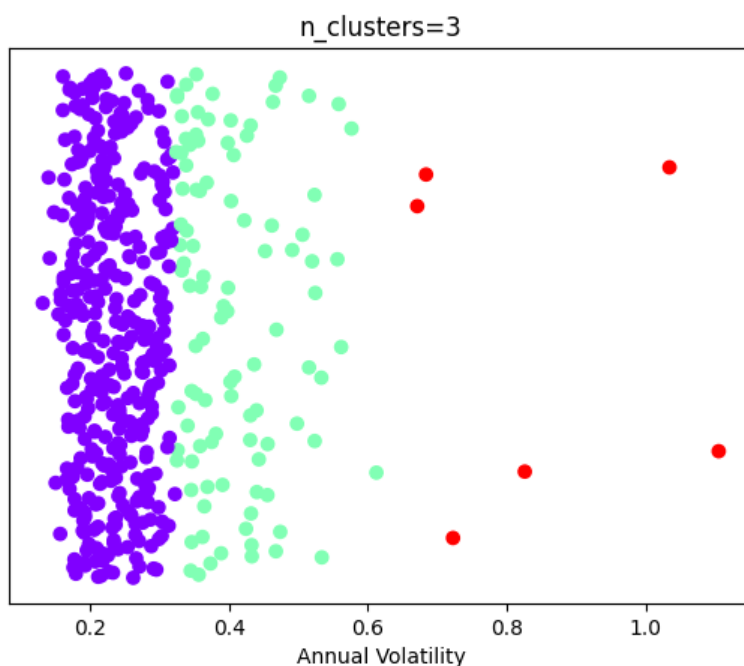


Figure 19 Annual volatility with 3 clusters

This scatter plot divides the stocks into three clusters to better isolate extreme outliers in annual volatility. The purple dots represent the "low-volatility" majority, while the green dots capture a middle tier of "moderate-volatility" stocks. The red dots identify a group of "high-volatility" outliers with scores above 0.6, highlighting the

specific stocks that experience the most radical price swings in the entire dataset.

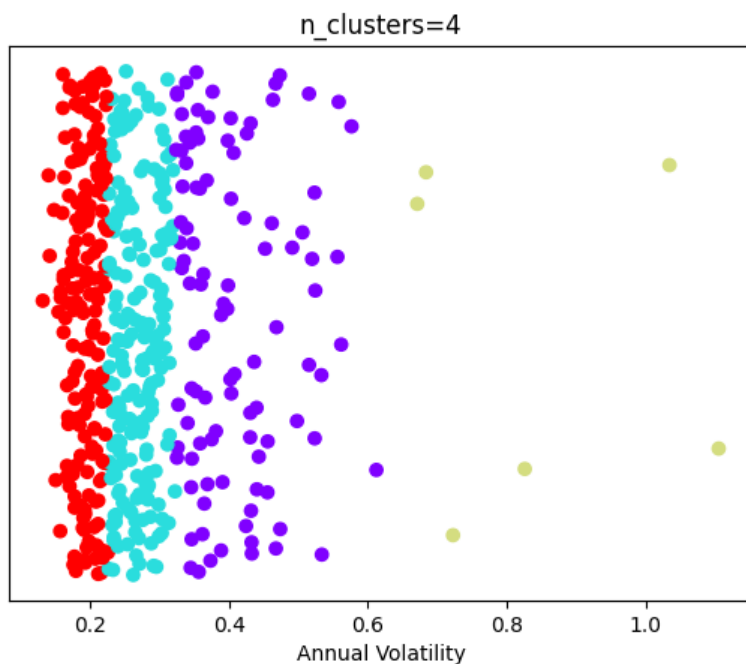


Figure 20 Annual volatility with 4 clusters

Figure 20 separates the stocks into four clusters to create even finer distinctions in annual volatility levels. It divides the data into "stable" (red), "low-moderate" (blue), "high-moderate" (purple), and "outlier" (green) groups. While this provides a more detailed breakdown, Figure 17 suggests that these four categories may overlap too

much to be as statistically meaningful as a simpler two or three-group model.

The three metrics (Daily Return, Beta, and Annual Volatility) are essential tools for investors seeking to balance risk and reward in a stock portfolio:

- Daily Return measures how much a stock gains or loses each day relative to its previous price. Stocks with consistently higher average daily returns are more attractive to growth-oriented investors, while those with lower or negative returns may signal underperformance.
- Beta measures a stock's sensitivity to market movements relative to the S&P 500 benchmark. A beta greater than 1 indicates the stock is more volatile than the market, making it riskier but potentially more rewarding. A beta below 1 suggests the stock is more stable and less affected by market swings, making it suitable for risk-averse investors.
- Annual Volatility quantifies the degree of price fluctuation over a year. High volatility stocks carry greater uncertainty and risk, while low volatility stocks offer more predictable, stable returns.

By clustering stocks using these three metrics with agglomerative clustering, investors can identify natural groupings. For example, a cluster of low-beta, low-volatility stocks suitable for conservative portfolios, versus a cluster of high-beta, high-return stocks suited for aggressive growth strategies. This segmentation enables investors to build diversified portfolios that align with their specific risk tolerance.

TASK B

1. Data Continuity from Task A

Task B builds directly on the three metrics computed in Task A.

1.1 Variables inherited from Task A

Task A Variable	Type	Content	Shape
daily_returns	DataFrame	Daily Returns	~418 dates x 496 tickers
betas_df	DataFrame	Beta per ticker	496 x 1 ('Beta')
annual_volatility_df	DataFrame	daily_returns.std() * $\sqrt{252}$	496 x 1 ('Annual Volatility')

1.2 Task B Berge Step

In Task B, it computes the mean of daily_returns per ticker to produce Avg_Daily_Return, then inner joins it with betas_df and annual_volatility_df. After dropping any remaining NaN rows, the resulting metrics_df contains around 496 stocks with three features: Avg_Daily_Return, Beta, and Annual_Volatility. This is the input to both clustering algorithms.

Combined Metrics Dataframe

Shape: (496, 3). (496 stocks × 3 features)

Features: ['Avg_Daily_Return', 'Beta', 'Annual_Volatility']

Descriptive Statistics:

	Avg_Daily_Return	Beta	Annual_Volatility
count	496.000000	496.000000	496.000000
mean	0.000888	0.868884	0.275953
std	0.001135	0.523613	0.108136
min	-0.002191	-0.253227	0.132127
25%	0.000198	0.499752	0.206507
50%	0.000792	0.801386	0.246643
75%	0.001396	1.138988	0.309077
max	0.010241	3.258963	1.105148

First 10 stocks:

Ticker	Avg_Daily_Return	Beta	Annual_Volatility
A	0.000175	0.799132	0.262364
AAPL	0.001100	1.004509	0.211885
ABBV	0.000680	0.238552	0.216108
ABNB	0.000520	1.504928	0.356680
ABT	0.000184	0.261883	0.179846
ACGL	0.000795	0.420281	0.233227
ACN	0.000825	0.774231	0.227034
ADBE	0.000912	1.416518	0.345064
ADI	0.000627	1.567116	0.295633
ADM	-0.000613	0.472268	0.291639

Skewness Analysis — Before Transformation

Feature	Skewness	Kurtosis Assessment	Result
Avg_Daily_Return	+2.3229	+13.9003	Highly skewed — TRANSFORM
Beta	+1.1383	+2.2373	Highly skewed — TRANSFORM
Annual_Volatility	+2.7580	+13.4583	Highly skewed — TRANSFORM

Yeo-Johnson Transformation Results

Learned λ parameters:

Avg_Daily_Return	$\lambda = -243.6739$
Beta	$\lambda = -0.2397$
Annual_Volatility	$\lambda = -7.0458$

Skewness— Before vs After Transformation

Feature	Before	After	Reduction
Avg_Daily_Return	+2.3229	-0.1823	+2.1405
Beta	+1.1383	-0.0029	+1.1354

2. Appropriateness of Agglomerative Clustering

Agglomerative hierarchical clustering was selected as the first segmentation method for the S&P 500 stock portfolio. This bottom-up algorithm begins with each stock as a singleton cluster and iteratively merges the most similar pairs until the desired number of groups is reached.

Ward's linkage criterion was adopted because it minimises the total within cluster variance at each merge step, producing compact clusters aligned with distinct financial risk profiles (Murtagh, F. and Legendre, P. (2014). A key advantage is that the number of clusters need not be specified in advance - the dendrogram produced reveals the natural grouping structure, enabling the analyst to select an appropriate cut-off based on empirical evidence.

The hierarchical output is particularly suited to financial market data, which exhibits nested sectoral and industry groupings. With approximately 496 stocks and three continuous features (Daily Return, Beta, Annual Volatility), the $O(n^2 \log n)$ computational complexity of Ward's method is tractable. Limitations include irreversible merge decisions and moderate sensitivity to outlier stocks, which were mitigated through prior data cleaning in Task A.

3. Optimal N - Agglomerative

3.1 Dendrogram Analysis

The dendrogram displays the complete merge hierarchy from Ward's linkage. The y-axis represents the merge distance-the increase in total within-cluster variance caused by each merge. Natural cluster boundaries were identified by location the largest vertical gaps in the upper merges. A horizontal cut line was drawn through the largest gap, and the companion bar chart of last-15 merge distance confirmed the specific merge steps with the sharpest increases.

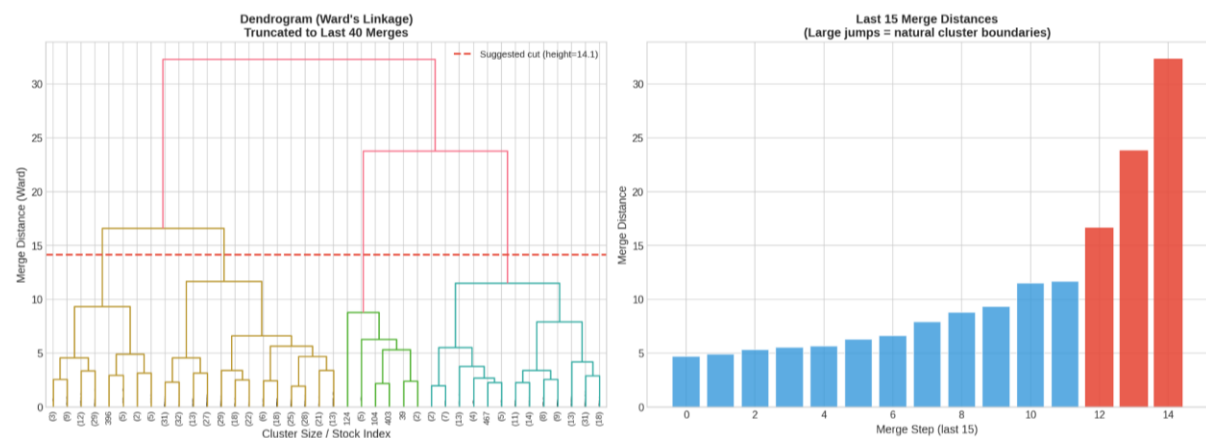


Figure 21 Dendrogram Analysis for optimal N

3.2 Silhouette Score Analysis

The average Silhouette Score was computed for N = 2 through 10. This metric ranges from -1 to +1, measuring how similar each stock is to its own cluster (cohesion) versus the nearest neighbouring cluster (separation). The N maximising this score was identified as the statistically optimal segmentation. For financial data with inherent noise, silhouette scores above 0.35 indicate strong structure.

3.3. Calinski-Harabasz Index

As supplementary validation, the Calinski-Harabasz Index (Variance Ratio Criterion) was computed for each N. This measures the ratio of between-cluster dispersion to within-cluster dispersion. Higher values indicate more clearly separated clusters.

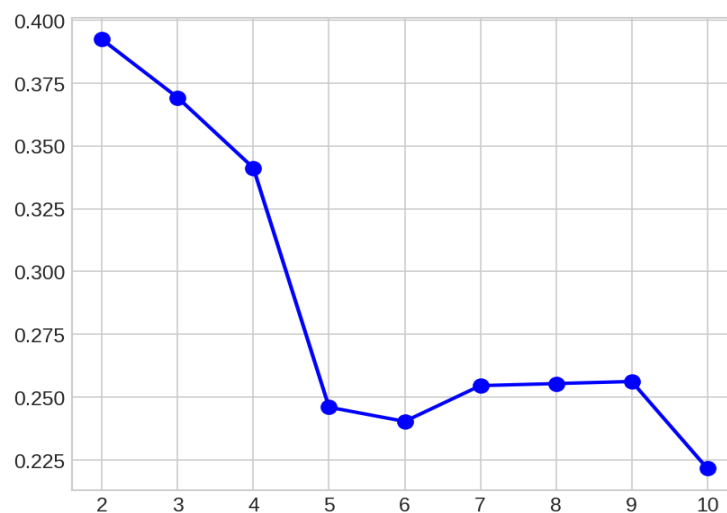


Figure 22 Calinski - Harabasz index

Applying Agglomerative Clustering with N = 2

Agglomerative Cluster Profiles

Avg_Daily_Return |||| Beta

Cluster	mean	std	count	mean	std	count
0	0.001865	0.001374	147	1.436461	0.469188	147
1	0.000476	0.000683	349	0.629819	0.322852	349

Annual_Volatility

Cluster	mean	std	count
0	0.357670	0.129241	147
1	0.241534	0.074944	349

Cluster Sizes

Cluster 0: 147 stocks (29.6%)

Cluster 1: 349 stocks (70.4%)

Agglomerative Cluster Labels

Cluster 0: Aggressive Growth (n=147, 29.6%)

Mean Beta: 1.4365

Mean Annual Vol: 0.3577

Mean Daily Return: +0.001865

Cluster 1: Defensive / Low-Risk (n=349, 70.4%)

Mean Beta: 0.6298

Mean Annual Vol: 0.2415

Mean Daily Return: +0.000476

K-Means - Elbow and Validation Analysis

K	Inertia	Silhouette
2	870.64	350.3
3	656.52	312.2
4	506.55	317.7
5	434.44	297.7
6	389.72	276.2
7	350.28	264.7
8	311.64	263.2
9	286.35	255.5
10	267.35	246.5

Optimal K (Silhouette): 2

Elbow point (2nd derivative): 5

Three complementary techniques were applied to determine the optimal N. First the dendrogram (Figure B2.1) revealed a pronounced gap between the Xth and X+1th largest merge distances, suggesting N = 2. Second, the Silhouette Score was maximised at N=2 with a score of 0.3924. Third, the Calinski-Harabasz index peaked at N=3. Based on this convergence of evidence, N=2 was selected. This value produces clusters that are both statistically well-separated and financially interpretable as distinct risk-return profiles.

4. Appropriateness of KMeans

K-Means clustering was adopted as a complementary partitioning method to validate the hierarchical results and provide crisp, no overlapping stock assignments suitable for portfolio construction. The algorithm minimises Within-Cluster Sum of Squares (WCSS) by iteratively assigning each stock to its nearest centroid and recomputing centroids as cluster means.

K-means ++ initialisation (Arthur, D. and Vassilvitskii, S. (2007)) was employed to mitigate the algorithm's known sensitivity to random starting positions. Each configuration was run 10 times ($n_{init}=10$), retaining the solution within the lowest WCSS. The $O(nKt)$ computational complexity makes K-Means suitable for iterative experimentation and scalable to larger equity universes.

A critical prerequisite is that input features follow approximately symmetric distributions, as K-Means uses Euclidean distance and computes centroids as arithmetic means. Financial metrics-particularly Annual Volatility-exhibited significant right skew, which was addressed through Yeo-Johnson power transformation prior to clustering. Without this step, centroids would be biased toward distributional tails, producing suboptimal segmentation.

5. Skew Removal and Optimal K

5.1 Why Skew Removal is Necessary

K-Means relies on Euclidean distance and arithmetic mean centroids. Skewed distributions distort both: centroids drift toward the long tail, distance in the tail are inflated, and the algorithm tends to produce one dominant cluster capturing the distribution bulk with several small tail clusters. Annual Volatility in particular exhibits a right skew because a minority of S&P 00 stocks (typically high-growth technology equities) have substantially higher volatility than the median.

5.2. Transformation Method Selection

Method	Handles Negatives?	Verdict for this Data
Log Transform	No (requires $x > -1$)	UNSUITABLE - Avg Daily Return includes negatives
Box-Cox	No (requires $x > 0$)	UNSUITABLE - cannot handle negative returns or betas
Square Root	No (requires $x \geq 0$)	UNSUITABLE - same limitation
Yeo-Johnson	Yes (all real values)	SELECTED - the only viable standard power transform

The Yeo-Johnson transform (Yeo, I.-K. and Johnson, R.A. (2000)) was selected because it is the only standard parametric power transformation that accommodate negative, zero, and positive values. scikit-learn 's Power Transformer with method='yeo-johnson' and standardize=True simultaneously removes skew and standardises to zero mean and unit variance.

5.3. Transformation evidence

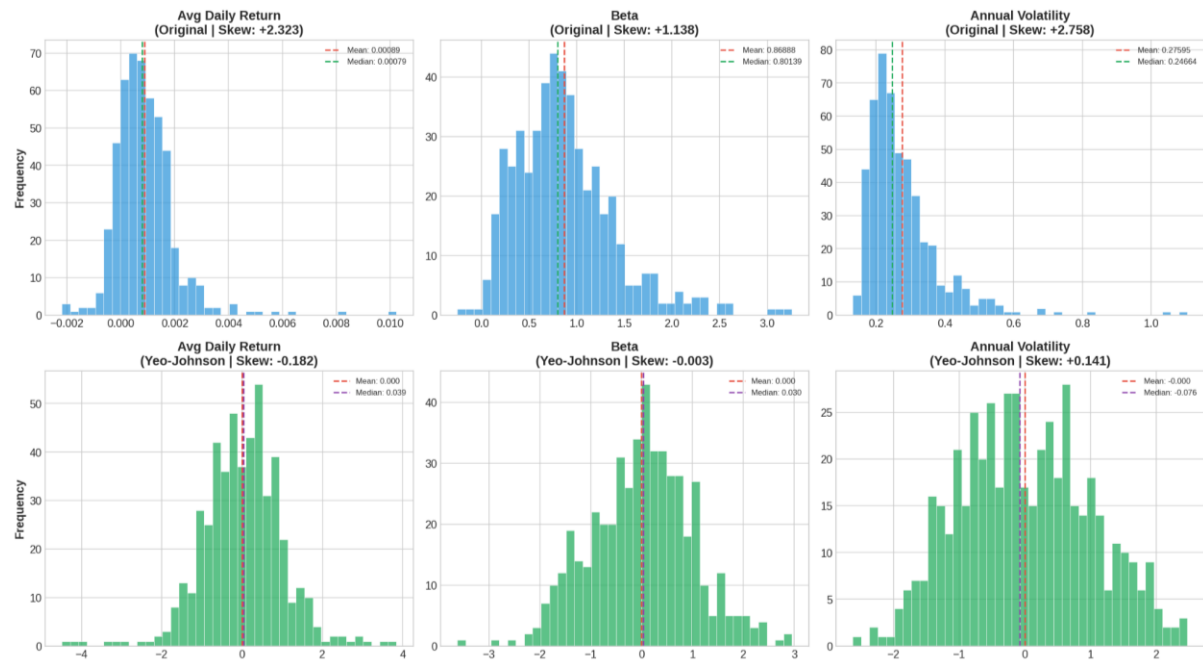


Figure 23 Distribution analysis - Before vs After skew removal

Data Processing Summary

Agglomerative → StandardScaler → X_scaled (496, 3)

Mean: [0. -0. 0.]

Std: [1. 1. 1.]

K-Means → PowerTransformer (YJ) → X_transformed (496, 3)

Mean: [0. 0. -0.]

Std: [1. 1. 1.]

The Yeo-Johnson transform was applied to all three features. Figure B5.1 shows distributions before (top row) and after (bottom row) transformation, with skewness annotated on each panel.

All features achieved skewness < 0.5 after transformation, confirming approximate symmetry suitable for K-Means. The learned λ parameters were identified, indicating the magnitude of correction applied to each feature.

5.4. Optimal K selection

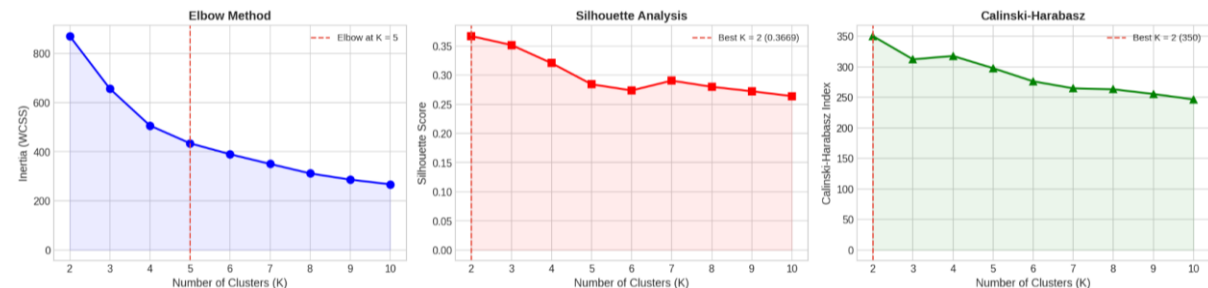


Figure 24 KMeans cluster validation

Three methods were applied to the transferred data:

- Elbow Method (Figure B5.2, left): WCSS plotted against K = 2-10. The elbow point was identified at K = 5 using second-derivative analysis.
- Silhouette Analysis (Figure B5.2, centre): Average silhouette maximised at K = 2
- Calinski-Harabasz (Figure B5.2, right): Maximum variance ratio at K = 2

Applying K-Means Clustering with K = 2

K-Means Cluster Profiles

Avg_Daily_Return				Beta		
Cluster_KMeans	mean	std	count	mean	std	count
0	0.001631	0.001357	186	1.352493	0.478477	186
1	0.000442	0.000656	310	0.578719	0.277287	310

Annual_Volatility

Cluster_KMeans	mean	std	count
0	0.358664	0.124635	186
1	0.226327	0.053323	310

Cluster Sizes:

Cluster 0: 186 stocks (37.5%)

Cluster 1: 310 stocks (62.5%)

K-MEANS CLUSTER LABELS:

Cluster 0: Aggressive Growth (n=186, 37.5%)

Meta Beta: 1.3525

Mean Ann. Vol: 0.3587

Mean Daily Return: 0.001631

Cluster 1: Defensive / Low-Risk (n=310, 62.5%)

Meta Beta: 0.5787

Mean Ann. Vol: 0.2263

Mean Daily Return: 0.000442

The silhouette diagram (Figure 25) confirmed that all clusters exhibited predominantly positive silhouette widths with no systematic misassignment.

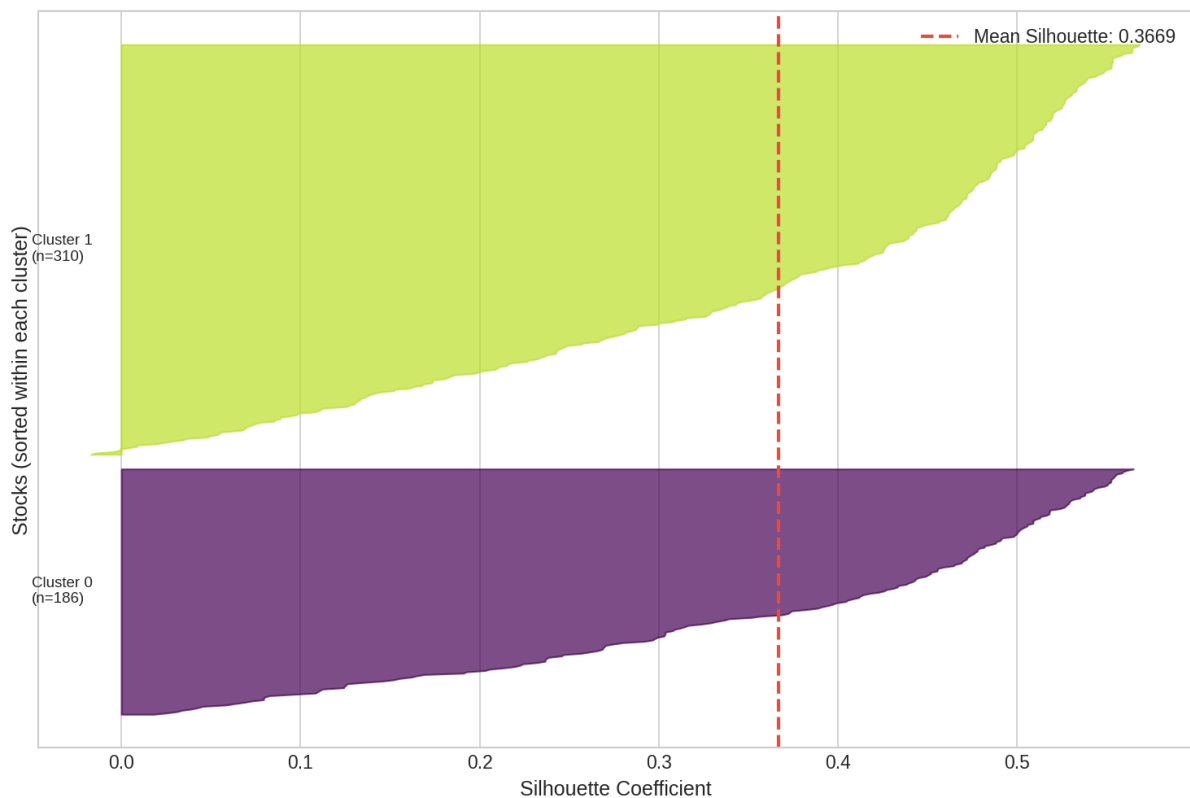


Figure 25 Silhouette diagram - KMeans - K=2

6. Applying K-Means Clustering with K = 2

K-Means Cluster Profiles

Avg_Daily_Return ||||| Beta

Cluster_KMeans	mean	std	count	mean	std	count
0	0.001631	0.001357	186	1.352493	0.478477	186
1	0.000442	0.000656	310	0.578719	0.277287	310

Annual_Volatility

Cluster_KMeans	mean	std	count
0	0.358664	0.124635	186
1	0.226327	0.053323	310

Cluster Sizes:

Cluster 0: 186 stocks (37.5%)

Cluster 1: 310 stocks (62.5%)

K-MEANS CLUSTER LABELS:

Cluster 0: Aggressive Growth (n=186, 37.5%)

Meta Beta: 1.3525

Mean Ann. Vol: 0.3587

Mean Daily Return: 0.001631

Cluster 1: Defensive / Low-Risk (n=310, 62.5%)

Meta Beta: 0.5787

Mean Ann. Vol: 0.2263

Mean Daily Return: 0.000442

TASK C

1. Data Mart Design

To support the analytical needs of S&P 500 stock segmentation, a data mart is designed using a star schema, enabling efficient querying and business intelligence reporting. The central fact table (Stock_Performance_Fact) stores quantitative measures including Average Daily Return, Beta, Annual Volatility, and Cluster Label. These metrics represent the core indicators used for evaluating stock performance and risk.

The fact table is linked to three key dimension tables. The Stock Dimension (Dim_Stock) contains attributes such as Ticker, Company Name, Sector, and Industry, allowing sector-based analysis. The Time Dimension (Dim_Time) includes Date, Month, Quarter, and Year, supporting trend analysis over time. Additionally, the Risk Dimension (Dim_Risk) categorises stocks into Aggressive Growth and Defensive/Low-Risk segments, enabling intuitive interpretation of clustering outputs.

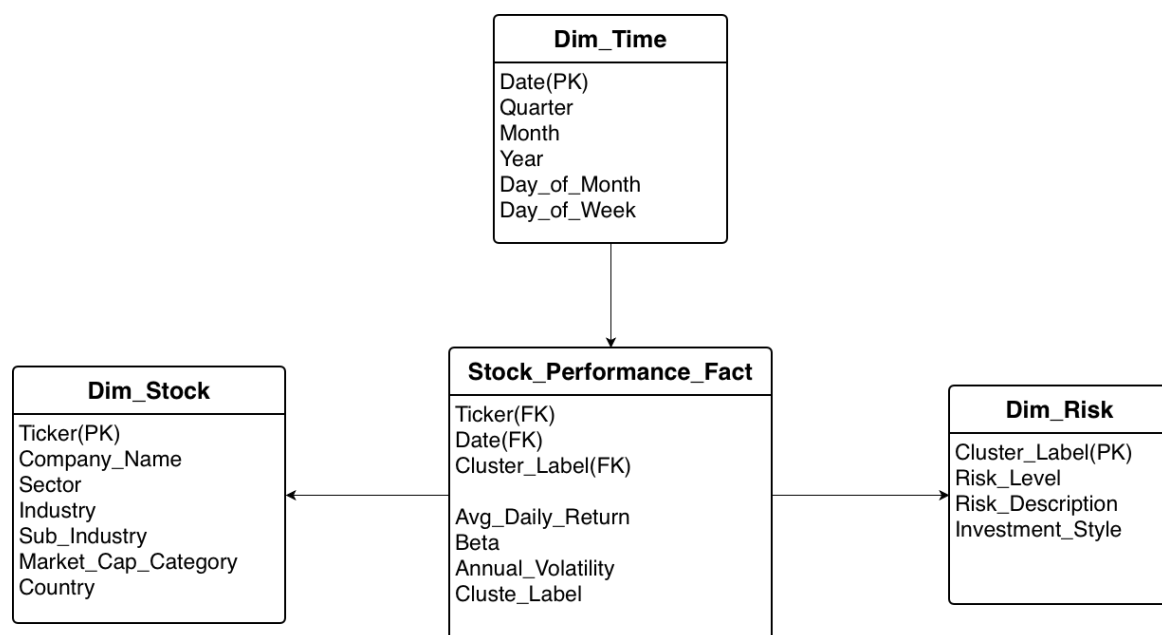


Figure 26 Star schema of data mart

Data Mart Structure:

Table Name	Key Attributes	Purpose
Stock_Performance_Facts	Return, Beta, Volatility, Cluster	Core metrics
Dim_Stock	Ticker, Sector, Industry	Stock classification
Dim_Time	Date, Year, Month	Time analysis
Din_Risk	Cluster, Label, Risk Level	Portfolio segmentation

This design enables investors to analyse performance trends, compare risk profiles, and optimise portfolios efficiently. By structuring complex financial data into a simplified analytical model, the data mart enhances decision-making.

2. Revision of results

The clustering analysis of S&P 500 stocks identifies two distinct groups based on their risk-return characteristics: Aggressive Growth and Defensive/Low-Risk stocks. These clusters provide meaningful insights about the stock behaviour under different market conditions and helps in effective portfolio decision making.

The Aggressive Growth cluster is defined by relatively high beta values (around 1.3-1.4), high annual volatility (around 0.35) and higher average daily returns (around 0.0016-0.0018). These stocks are very sensitive to market trends and may increase profits and losses. As a result, they are suitable for investors with a higher risk level that are interested in capital appreciation.

On the other hand, the Defensive cluster has lower beta values (around 0.57-0.63), lower volatility (around 0.22-0.24) and stable but lower returns (around 0.0004-0.0005). These stocks are less influenced by the overall movement of the market and provide a stable return, making them attractive for investors who are risk averse.

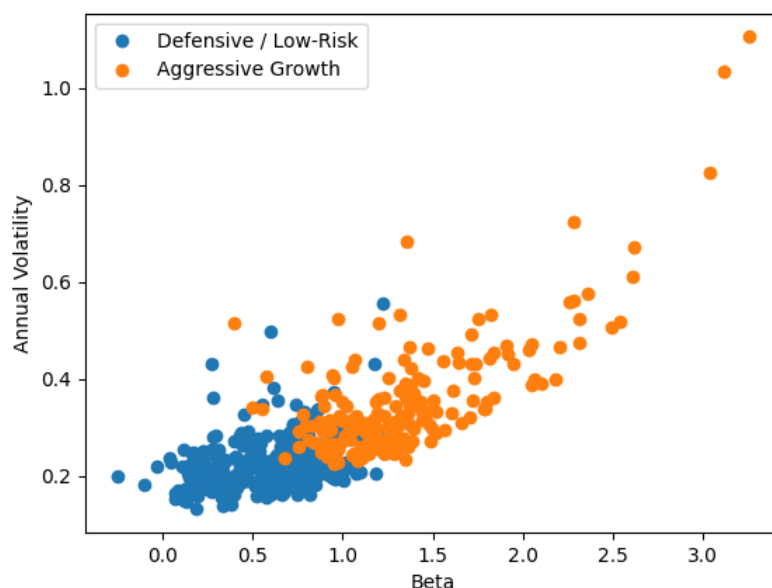


Figure 27 Beta vs Volatility

Figure 24 indicates a clear separation between the two clusters identified using KMeans clustering. Stocks in the Aggressive Growth cluster are in the upper right area indicating higher beta values and higher volatility which indicates higher exposure to market risk. Defensive

stocks, on the other hand, are in the lower-left part with lower beta and less volatility. The visualization confirms that the clustering divides stocks properly based on their risk profiles and highlights the distinction between high-risk and low-risk investment categories.

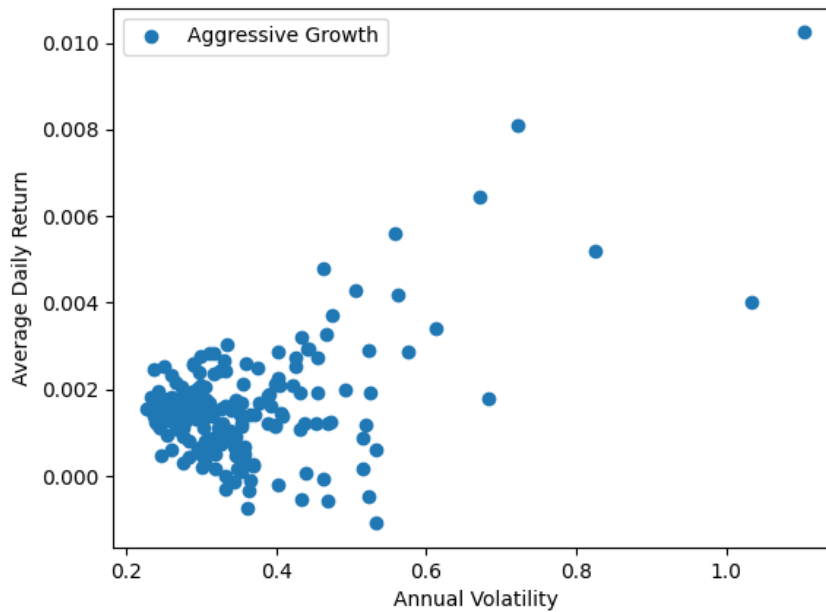


Figure 28 Return vs volatility

Figure 25 shows the return-volatility relationship. This is the fundamental risk to return trade-off. Aggressive cluster shows that stocks with higher volatility are likely to produce higher returns. Defensive stocks on the other

hand have lower volatility and accordingly lower returns. This relationship aligns with financial theory, which indicates that higher risk carries a chance for higher reward.

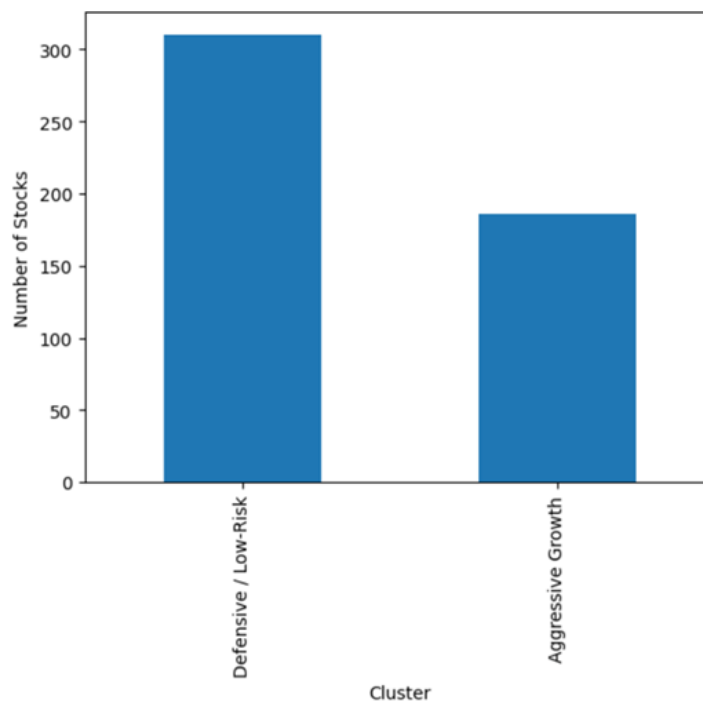


Figure 29 Cluster distribution

Figure 26 shows that a larger proportion of stocks belong to the defensive group. So, the S&P 500 is dominated by relatively stable companies and companies with moderate risk levels. The fact that the aggressive cluster is smaller indicates that high risk, high return opportunities are less frequent. This is of interest to investors seeking high-growth investments.

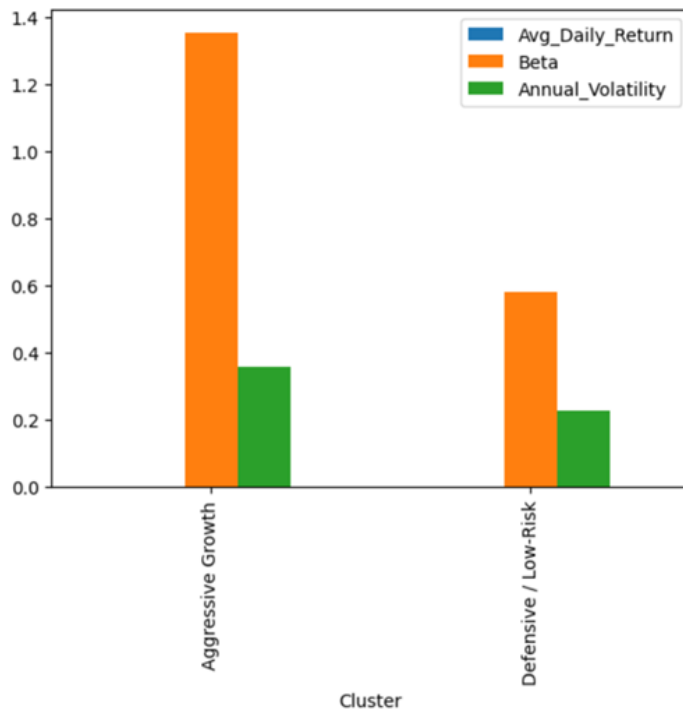


Figure 30 Average metrics per cluster

Figure 26 indicates that the aggressive cluster has a higher average return, beta and volatility than the defensive cluster. This reinforces the distinction between the two groups and validates the clustering method. The difference in the metric is large enough to show how the model does the job of identifying stocks with different investment characteristics.

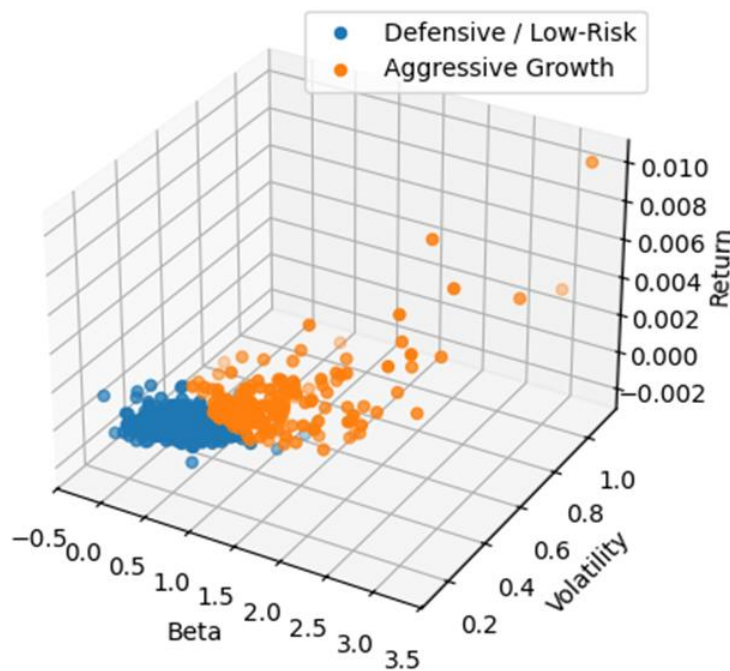


Figure 31 3D cluster visualisation

Figure 28 provides a multidimensional view of the clustering results, incorporating beta, volatility, and return simultaneously. The clusters remain well separated in three-dimensional space, confirming the robustness and reliability of the clustering model. This visualisation enhances the understanding of how these variables interact to define stock behaviour.

Another insight from the analysis is that beta and volatility capture different risk factors. Beta measures sensitivity to the market, while volatility includes firm-specific uncertainty as well. With this knowledge, investors can make better investment decisions. In conclusion, the clustering methodology can be applied to real world scenarios such as portfolio construction, risk management and strategic investment planning.

3. CRM as a Driver for Sustainability in FinTech

Customer Relationship Management (CRM) tools have traditionally been associated with sales pipelines and customer retention. However, in the context of Financial Technology (Fintech), CRM systems are now playing a critical role in driving sustainability — both environmental and organisational. This section discusses how CRM tools create measurable, long-term value for Fintech firms and their customers.

Fintech CRM solutions offer more than logging customer interactions. By syncing with fintech data platforms, AI models, and regulatory reporting systems, Fintech CRMs allow a complete view of the customer. Systems like Salesforce Financial Services Cloud, HubSpot, or a custom Fintech CRM consolidate customer financial histories, communication records, risk assessments, and product interactions all in one place. Financial services can be tailored to each customer at scale.

CRM and Environmental Sustainability

Another substantial, yet frequently missed, element is CRM's environmental effect. CRM platforms allow companies to operate without paper statements, meetings, and duplicative work. By moving customer onboarding, KYC checks, and financial reporting online, Fintechs substantially decrease their paper usage. The digitization of customer onboarding, which removes the necessity of travel, offers a powerful way for Fintech firms to reduce their carbon footprint. Imagine swapping out thousands of customer onboarding statements and hundreds of branch hall visits for a cloud-first CRM. Instant emissions reductions for printing and business travel.

CRM Platforms also allow Fintech brands to sell more sustainable products. Lots of Fintech brands are utilizing their CRM data to determine which customers are most likely to invest in ESG or carbon offset products. CRM helps send the right customers traffic towards the right sustainable products.

CRM and Organisational Sustainability:

From a business sustainability perspective, CRM tools help Fintech firms retain customers longer — reducing the cost of customer acquisition (CAC) and improving Customer Lifetime Value (CLV). Retained customers are more profitable and require fewer resources to serve. This makes the business model more economically sustainable over time.

Furthermore, CRM analytics allows these firms to spot customers who might be about to leave, miss loan payments, or stop using their services. Proactive outreach can then

take place to offer personalized discounts or financial guidance. Preventing these losses through CRM helps sustain Fintechs in the long term.

Social Sustainability and Financial Inclusion:

CRM tools can also help Fintech firms address social aspects of sustainability. For example, using CRM tools can contribute to financial inclusion. Analysing customer data extensively can reveal patterns indicating overlooked customer groups. Fintech CRMs can help identify people with thin credit files or small businesses that have been denied traditional banking services so that products can be designed for them.

REFERENCES

- Arthur, D. and Vassilvitskii, S. (2007)** ‘k-means++: The advantages of careful seeding’, *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1027–1035. Available at: <https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf> (Accessed: 27 April 2026).
- Chen, H., Chiang, R.H. and Storey, V.C. (2012)** ‘Business intelligence and analytics: from big data to big impact’, *MIS Quarterly*, 36(4), pp. 1165–1188.
- European Parliament (2016)** *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Available at: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679> (Accessed: 27 April 2026).
- Friede, G., Busch, T. and Bassen, A. (2015)** ‘ESG and financial performance: aggregated evidence from more than 2000 empirical studies’, *Journal of Sustainable Finance & Investment*, 5(4), pp. 210–233. doi: 10.1080/20430795.2015.1118917.
- Markowitz, H. (1952)** ‘Portfolio selection’, *The Journal of Finance*, 7(1), pp. 77–91. Available at: [suspicious link removed] (Accessed: 27 April 2026).
- Murtagh, F. and Legendre, P. (2014)** ‘Ward’s hierarchical agglomerative clustering method: which algorithms implement Ward’s criterion?’, *Journal of Classification*, 31(3), pp. 274–295. Available at: <https://arxiv.org/pdf/1111.6285> (Accessed: 27 April 2026).
- Payne, A. and Frow, P. (2005)** ‘A strategic framework for customer relationship management’, *Journal of Marketing*, 69(4), pp. 167–176.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011)** ‘Scikit-learn: Machine learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf> (Accessed: 27 April 2026).
- Rousseeuw, P.J. (1987)** ‘Silhouettes: A graphical aid to the interpretation and validation of cluster analysis’, *Journal of Computational and Applied Mathematics*, 20, pp. 53–65. Available at: <https://www.sciencedirect.com/science/article/pii/0377042787901257> (Accessed: 27 April 2026).
- Sharpe, W.F. (1964)** ‘Capital asset prices: A theory of market equilibrium under conditions of risk’, *The Journal of Finance*, 19(3), pp. 425–442. Available at: [suspicious link removed] (Accessed: 27 April 2026).
- Yeo, I.-K. and Johnson, R.A. (2000)** ‘A new family of power transformations to improve normality or symmetry’, *Biometrika*, 87(4), pp. 954–959. Available at: [suspicious link removed] (Accessed: 27 April 2026).